

AgentGuard: An Attribute-Based Access Control Framework for Tool-Use LLM-Based Agent

Jiaqi Luo^{†*}, Songyang Peng^{†*}, Jiarun Dai^{†§}, Zhile Chen[†], Zhuoxiang Shen[†],
Geng Hong[†], Xudong Pan^{†‡}, Yuan Zhang[†], Min Yang^{†§}

[†]Fudan University [‡]Shanghai Innovation Institute

[§] Co-corresponding authors

Abstract

LLM-based agents have recently attracted significant attention due to their ability to autonomously invoke relevant tools to accomplish complex tasks. However, recent studies have shown that these agents face severe security risks, which may lead to privacy leakage, financial loss, or even full system compromise. In this paper, we present AgentGuard, an attribute-based access control framework for tool-use LLM-based agents. AgentGuard adopts a client-server architecture. On the client side, AgentGuard provides lightweight integration for agents implemented in different programming languages and architectures. It requires only minor code modifications (e.g., around 10 lines) without changing the underlying agent execution logic. On the server side, AgentGuard provides three complementary inspection mechanisms to cover both single-tool and cross-tool security risks in agent execution. In addition, it offers a visualized front-end interface for security policy specification and runtime auditing. Currently, AgentGuard is publicly accessible at <https://github.com/WhizardAgent/AgentGuard>.

1 Introduction

LLM-based agents have recently attracted significant attention due to their ability to autonomously invoke relevant tools to accomplish complex tasks [19, 22]. Mainstream agent applications, such as Codex [15], and OpenClaw [16], are attracting millions of users. However, recent studies [6, 11, 13, 24] have revealed that such agents face severe security risks, which may ultimately result in privacy leakage, financial loss, or even full system compromise.

Research Gap. In response to these threats, a large body of research has sought to improve the security of LLM-based agents [9]. Generally, existing approaches fall into two categories: *model-level* defenses and *system-level* defenses. ❶ At the model level, prior approaches [4, 8, 12] attempt to enhance the robustness of the underlying models in agents through prompt engineering and fine-tuning. However, recent studies [20, 23] have shown that defenses relying solely on improving model capabilities can still be bypassed by adaptive attacks. ❷ At the system level, some works [10, 18, 21] leverage auxiliary LLMs to detect and intercept malicious intentions during agent execution. Other approaches [5, 7, 17] attempt to ensure security by enforcing predefined security policies. Nevertheless, as summarized in Table 1, these solutions still suffer from insufficient risk coverage, poor compatibility with existing agents, and complex policy specification.

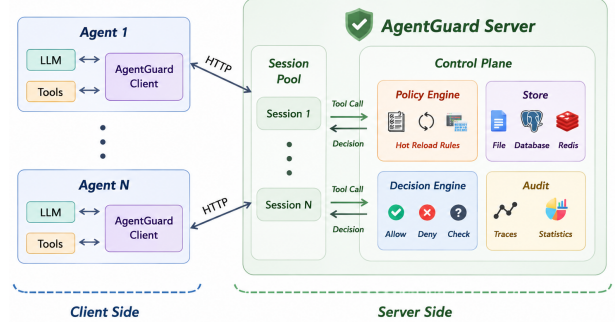


Figure 1: Architecture of AgentGuard.

Our Work. To bridge these gaps, we present **AgentGuard**, a mandatory access control framework for LLM-based agents. In general, AgentGuard adopts a *client-server architecture*, where the two components communicate through a network. ❶ On the client side, AgentGuard provides lightweight integration for agents implemented in different programming languages and architectures. It requires only minor code modifications (e.g., around 10 lines) without changing the underlying agent execution logic. During runtime, the AgentGuard client monitors each tool invocation, forwards the relevant contextual information to the server, and enforces the server’s decision to either allow or deny the action. ❷ On the server side, AgentGuard provides a visualized front-end interface that allows users to flexibly configure security policies through intuitive parameterized forms. During runtime, the server receives tool-related information from the client and performs inspection through three complementary mechanisms, i.e., rule-based detection, LLM-assisted detection, and manual verification, to cover both single-tool and cross-tool security risks in agent execution.

Contributions. Our contributions are summarized as follows:

- We propose AgentGuard, an attribute-based access control for tool-use LLM-based agent. AgentGuard adopts a lightweight client-server architecture that can be seamlessly integrated into existing agents, without modifying underlying logic.
- We develop the AgentGuard as a centralized security service with audit-oriented observability. The server can manage and inspect multiple concurrent agent sessions, while supporting visualized policy specification, runtime monitoring, and security auditing.
- To support future research and community development on secure LLM-based agents, we publicly release AgentGuard at <https://github.com/WhizardAgent/AgentGuard>.

* These authors contributed equally.

Email: {jqluo24, sypeng23, zhilechen25, zxshen22}@m.fudan.edu.cn; {jrdai, ghong, xdpn, yuanxzhang, m_yang}@fudan.edu.cn

Table 1: Comparison with existing works.

Approach	Risk Type		Intervention Phase		Decision Mechanism			Compatibility		Usability			
	Single-tool	Cross-tool	Pre-execution	Post-execution	Rule-based	LLM-based	Manual Verify	Framework-agnostic	Language-agnostic	Agent-decoupled	Low-intrusion	Visual Rule Setup & Audit	Dynamic Rule Update
AgentDoG [1]	✓	✓	✓			✗ ¹		✓					
ClawSentry [2]	✓	✗ ²	✓	✗ ³	✗ ²	✗ ¹	✓	✓	✓	✓	✓		
AgentBound [3]	✓		✓		✓								
CaMeL [5]	✓	✓	✓		✓			✓			✓		
SECAgent [7]	✓	✓	✓		✓			✓					
ACE [10]	✓	✓	✓			✗ ¹							
Agent Gov. Toolkit [14]	✓		✓	✗ ³	✓	✗ ¹		✓	✓	✓	✓		✓
Progent [17]	✓	✓	✓		✓			✓			✓		
AgentSpec [18]	✓		✓		✗ ⁴	✗ ¹	✓	✓					
IsolateGPT [21]	✓	✓	✓			✗ ¹	✗ ⁵						
AgentGuard (Ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

✓ indicates full support, ✗ indicates partial support, and blank entries indicate no support.

¹ Existing LLM-based verification approaches rely on a unified prompt, preventing users from customizing prompts for specific scenarios or security requirements.

² ClawSentry cannot effectively extend security policies for multi-tool scenarios. Moreover, when a specific multi-tool execution flow is detected, it relies solely on LLM-based judgment to determine whether the action should be blocked, rather than enforcing user-defined security policies.

³ In both ClawSentry and Agent Gov. Toolkit, post-execution rules are hard-coded. Supporting new rules requires manual code implementation.

⁴ AgentSpec allows users to define custom security policies, but it requires substantial amounts of manual code implementation of rule-specific detection logic.

⁵ IsolateGPT only triggers human review at the LLM's request, preventing user-defined verification conditions.

2 AgentGuard

2.1 Overview

As illustrated in Figure 1, AgentGuard is an attribute-based access control for tool-use LLM-based agent that adopts a lightweight client-server architecture.

- **Client.** AgentGuard offers SDKs for multiple programming languages and requires only minimal code modifications (i.e., approximately 10 lines) to integrate with existing agent frameworks without altering the underlying execution logic. During runtime, the client monitors each tool invocation, forwards the relevant contextual information to the server, and enforces the server's decision to either permit or deny the requested action.
- **Server.** AgentGuard provides a visualized front-end interface that enables users to flexibly configure security policies through intuitive parameterized forms. During runtime, the server receives information from the client and performs security inspection through three complementary mechanisms, namely rule-based detection, LLM-assisted detection, and manual verification. AgentGuard can address both single-tool and cross-tool security risks during agent execution.

As summarized in Table 1, compared with existing defense approaches, AgentGuard provides broader coverage of security risks, supports more flexible decision-making mechanisms, and can be seamlessly integrated into existing agent frameworks across different programming languages and architectures. Furthermore, AgentGuard offers a visualized front-end interface that improves usability and simplifies security policy management.

2.2 Usage

Users can complete the deployment of AgentGuard and start using it in just four steps: launching the control server, performing

agent-side setup, configuring access control policies, and monitoring agents in runtime through the visualized front-end.

Launch of the Control Server. The control server is the central component of AgentGuard. It receives runtime events from agents, evaluates them with the active policies, and returns access control decisions. This separates security control from the agent's own planning and task logic. It also allows multiple agents on different machines to be managed under one policy view.

Agent-side Setup. On the agent side, AgentGuard is inserted between the agent framework and its tools. The agent provides principal information such as identity, session, role, and trust level. During execution, each tool call is checked by the control server before the final action is taken. Adapters are provided for several mainstream agent frameworks, such as LangChain, AutoGen, and OpenAI Agents SDK, allowing users to integrate AgentGuard with minimal code and without modifying framework internals or heavily refactoring existing agents.

Configuration of Policies. AgentGuard uses declarative policies to describe when a tool call should be allowed, denied, or sent for review. A policy can refer to the agent identity, tool attributes, tool arguments, target addresses, and session history. This allows both simple checks on a single tool call and broader checks over a sequence of actions. Policies can be written in the DSL or configured through the visualized front-end.

Runtime Monitoring via Visualized Front-end. The visualized front-end gives users a runtime view of connected agents, their tools, recent activity, and pending review requests. For each relevant tool call, users can inspect the matched rules, final decision, and audit metadata. This helps explain why an operation was allowed, denied, or escalated. The same interface can also be used to configure policies and refine them based on audit records.

2.3 Access Conditions

AgentGuard has been publicly released at <https://github.com/WhizardAgent/AgentGuard> under the MIT License. It is freely available for academic research, educational purposes, and other non-commercial or commercial applications permitted by the license. We kindly request that any use of AgentGuard in academic publications or derived projects appropriately acknowledge the project and cite our paper when applicable.

2.4 Future Plan

In future work, we plan to extend AgentGuard to support more mainstream agent frameworks, enable protection for multi-agent scenarios, and incorporate monitoring of LLM inputs and outputs with automatic security policy recommendations tailored to different agent implementations. We also warmly welcome contributions from the community to help maintain and improve this framework. Please feel free to submit issues and pull requests.

3 Conclusion

In this paper, we present AgentGuard, an attribute-based access control for tool-use LLM-based agent. AgentGuard can be seamlessly integrated into existing agent applications, while supporting the detection and mitigation of a wide range of security risks through multiple complementary auditing mechanisms. Currently, AgentGuard can be accessed through the website: <https://github.com/WhizardAgent/AgentGuard>.

References

- [1] AI45Lab. 2026. AgentDoG on Github. (2026). <https://github.com/AI45Lab/AgentDoG>
- [2] AI45Lab. 2026. ClawSentry on Github. (2026). <https://github.com/AI45Lab/ClawSentry>
- [3] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. 2026. AgentBound: Securing Execution Boundaries of AI Agents. In *Proceedings of the 34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE)*, Vol. 3. 24.
- [4] Sizhe Chen, Arman Zharmagambetov, Saeed Mahlouljifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2025. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2833–2847.
- [5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813* (2025).
- [6] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems* 37 (2024), 82895–82920.
- [7] Zimo Ji, Daoyuan Wu, Wenyuan Jiang, Pingchuan Ma, Zongjie Li, Yudong Gao, Shuai Wang, and Yingjiu Li. 2026. Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework. *arXiv preprint arXiv:2601.11893* (2026).
- [8] Changyue Jiang, Xudong Pan, and Min Yang. 2025. Think twice before you act: Enhancing agent behavioral safety with thought correction. *arXiv preprint arXiv:2505.11063* (2025).
- [9] Juhee Kim, Wenbo Guo, Dawn Song, UC Berkeley, and UC Santa Barbara. 2026. SoK: Attack and Defense Landscape of Agentic AI Systems. In *35th USENIX Security Symposium (USENIX Security 26)*.
- [10] Evan Li, Tushin Mallick, Evan Rose, William Robertson, Alina Oprea, and Cristina Nita-Rotaru. 2025. Ace: A security architecture for llm-integrated app systems. *arXiv preprint arXiv:2504.20984* (2025).
- [11] Fengyu Liu, Yuan Zhang, Jiaqi Luo, Jiarun Dai, Tian Chen, Letian Yuan, Zhengmin Yu, Youkun Shi, Ke Li, Chengyuan Zhou, et al. 2025. Make agent defeat agent: Automatic detection of {Taint-Style} vulnerabilities in {LLM-based} agents. In *34th USENIX Security Symposium (USENIX Security 25)*. 3767–3786.
- [12] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. 2025. Datasentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2190–2208.
- [13] Jiaqi Luo, Jiarun Dai, Fengyu Liu, Songyang Peng, Youkun Shi, Tong Bu, Geng Hong, Xudong Pan, and Yuan Zhang. 2026. Autonomy Comes with Costs: Detecting Denial-of-Service Vulnerabilities Caused by Resource Abusing in LLM-based Agents. In *35th USENIX Security Symposium (USENIX Security 26)*.
- [14] Microsoft. 2026. Agent Governance Toolkit on Github. (2026). <https://github.com/microsoft/agent-governance-toolkit>
- [15] OpenAI. 2026. The Official Website of Codex. (2026). <https://openai.com/codex/>
- [16] Openclaw. 2026. Openclaw on Github. (2026). <https://github.com/openclaw/openclaw>
- [17] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable privilege control for llm agents. *arXiv preprint arXiv:2504.11703* (2025).
- [18] Haoyu Wang, Christopher M Poskitt, and Jun Sun. 2025. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. *arXiv preprint arXiv:2503.18666* (2025).
- [19] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345.
- [20] Yifei Wang, Dizhan Xue, Shengjie Zhang, and Shengsheng Qian. 2024. Badagent: Inserting and activating backdoor attacks in llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9811–9827.
- [21] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2024. Isolategpt: An execution isolation architecture for llm-based agentic systems. *arXiv preprint arXiv:2403.04960* (2024).
- [22] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. 2025. The rise and potential of large language model based agents: A survey. *Science China Information Sciences* 68, 2 (2025), 121101.
- [23] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. 2025. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 7101–7117.
- [24] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. 10471–10506.